SOFTWARE DESIGN DOCUMENT FOR THE “SHERLOCK” PROGRAM

1. INTRODUCTION

1.1 Introduction

This document contains the Software Requirements Specifications (SRS), methodology analysis and architectural design for the software application “Sherlock”; which has as an objective, to facilitate an AI driven assistant system for detectives who are in need of a more efficient way of iterating through case files, while maintaining case integrity and information privacy secure.

1.2 Problem definition

A case is presented: The lead detective has a problem, all the thousands of file cases collected, witness statements and clues from all over the years are stored as digital text documents. A detective’s job is to be able to present necessary evidence when the opportunity comes, yet, having such a vast library of files can naturally make it difficult for him/her to remember specific details.

Searching through each file manually can be a tedious task, as it will be very time consuming. Especially considering the way in which each file is stored and separated (yearly folders, location folders, etc.).

Another possibility would be for the detective to discard old files. Overpopulating the library of files, specifically, with cases that have already been closed and solved makes this issue more complex. However, the usual detective will normally have an archive of cases, regardless of their status. There is a need for a tool that will solve this problem, an assistant that can simplify the throughout search of specific parameters/questions prompted by the detective while maintaining the integrity of the information.

1.3 Solution approach

The primary objective of this project is to develop a standalone, secure Traditional Desktop GUI Application programmed for investigative workflows. The system will empower detectives to securely authenticate, input private document libraries into a local knowledge base, and query their data using natural language. A core requirement of the system is absolute data integrity; the application must strictly prevent hallucinations and ensure all generated insights are directly verifiable against the provided source documents.

2. PROJECT DEFINITION

2.1 Objectives and functionality

This system's functionality will be divided into three subsystems: The frontend Subsystem (UI), the Backend API subsystem (orchestration and logic) and the RAG and Data Subsystem (The AI). Each of the subsystems has a responsibility within the implementation of the application, serving as different hot spots for the full integration of the entire system.

2.2 Project and application objectives

The main objective of project **"Sherlock"** is to build a digital assistant that enables a lead detective to instantly extract insights (such as suspect alibis, motives, and timelines) from thousands of pages of historical case files without manual re-reading. To guarantee absolute data integrity in an investigative context, the system must leverage a strict Retrieval-Augmented Generation (RAG) pipeline that eliminates AI hallucinations. So, if a user's question cannot be explicitly verified by the uploaded text files or PDFs, the application must deterministically fail-safe and respond with a strict boundaries statement: "I don't have enough evidence to answer that."

Architecturally, the application will be engineered as a decoupled, Python-based solution delivered via a single Docker container. It will connect a user-friendly frontend interface with a backend REST API, utilizing an API Key from language model which should be limited in terms of token usage, (Open AI) to ensure cost-efficiency.

2.3 Frontend Subsystem (UI)

This subsystem serves as the presentation layer that the detective interacts with. The subsystem will be connected to the API interface.

As for its responsibilities, it will handle file upload (PDF/Text), chat interface for asking questions, rendering the responses that passed the integrity test, displaying an error message when there is not enough evidence to provide an answer.

The system will use Streamit to handle HTTP requests for the backend API.

There will be several pages which the users can navigate through.

i. The welcome page will display a logo of the application.
ii. Home page will have a text box in which the users can ask the LLM their respective inquiries, in this page, the answers should also be displayed.
iii. Home page will also have interactive buttons (as a side menu) in which the user can submit their library of cases.
iv. It should be scrollable.
v. Upon clicking the side menu, the user will have several options to choose, list of cases and quick submission of cases.

vi. Upon clicking the list of cases, the user will be able to see all the cases they have submitted, plus being given the option to either delete or add new ones to the catalog.

2.4 Backend Subsystem (Orchestration and Logic)

This subsystem acts as the bridge between the user interface and the AI Intelligence. It will expose the REST endpoints required by the prompt.

It has the responsibility of receiving the files from the front end and passing them to the RAG pipeline. POST/query will receive the detective's question, invoke the RAG pipeline and return the bounded answer.

The system will use FastAPI, since it can automatically generate OpenAPI documentation(/docs).

When it comes to the submission and handling of the case files, the system will handle the process as follows:

i. Users submit a pdf/text file containing a case.
ii. The system will scan the document and save the most important credentials (such as the case subject, date and a generated ID) in a table.
iii. Upon hovering each case in the list of cases from the side menu, such credentials should be displayed.
iv. The system should signal an error message if a duplicate is found.

2.5 RAG & Data subsystem

This subsystem contains an engine that handles data ingestion, storage, and LLM prompting. It is kept separated from the API routes to make it easier to do Unit Tests.

Its responsibilities are processing and parsing PDFs/Text files to chunk them into readable segments. Then, it should embed those chunks and store them locally (Using ChromaDB). Finally, the system prompt for the LLM (In this via OpenAI API Key).

3. SYSTEM REQUIREMENTS CATALOG

Below, I list the set of software requirements, starting with functional requirements which are grouped into their respective subsystems, followed by the non-functional requirements, grouped by functionality.

3.1 Functional Requirements

3.1.1 Frontend Subsystem:

- ✓ **FR-1.1: User Welcome Interface:** The welcome page shall display the application logo.
- ✓ **FR-1.2: Sidebar Navigation:** Upon logging in, a persistent side menu shall allow the user to navigate between the Home Page, List of Cases, and Quick Case Submission.
- ✓ **FR-1.3: Document Upload:** The UI shall provide a file picker allowing the detective to upload PDF and `.txt` case files.
- ✓ **FR-1.4: Case Library Dashboard:** The user shall be able to view a scrollable list of all submitted cases, hover over a case to see its core details (Subject, Date, ID) and delete individual cases.
- ✓ **FR-1.5: Interactive Chat Feed:** The main page should provide a text input box for asking questions and a vertically scrollable chat window displaying the history of questions and answers.
- ✓ **FR-1.6: Empty Evidence Warning:** If the system cannot find the answer in the uploaded documents, the chat UI must display the exact message: *"I don't have enough evidence to answer that."*

3.1.2 Backend Subsystem:

- ✓ **FR-2.1: Case Document Ingestion:** The backend shall expose a `POST /documents` endpoint to accept files from the UI, reject duplicates, automatically extract metadata (Subject, Date, generated ID), and save these records to a local database table.
- ✓ **FR-2.2: Case Inventory Management:** The backend shall expose `GET` and `DELETE` endpoints to retrieve the full catalog of case metadata for the UI list or permanently delete a case from storage.
- ✓ **FR-2.3: Query Routing:** The backend shall expose a `POST /query` endpoint that accepts the detective's question, routes it to the AI pipeline, and returns the final bounded response.
- ✓ **FR-2.4: Interactive API Docs:** The API layer shall automatically generate a live documentation page (accessible at `/docs`) to test all available endpoints.

3.1.3 RAG & Data Subsystem

- ✓ **FR-3.1: Text Extraction & Chunking:** The system shall extract raw text from uploaded PDFs and text files, and split it into small, overlapping text blocks to preserve context.
- ✓ **FR-3.2: Local Vector Indexing:** The system shall convert text blocks into mathematical meanings (embeddings) and store them locally in ChromaDB, tagged with their respective Case IDs.
- ✓ **FR-3.3: Semantic Context Retrieval:** When a question is received, the engine shall search ChromaDB and pull out only the text blocks most relevant to the query's meaning.
- ✓ **FR-3.4: Strict AI Hallucination Guard:** The engine shall send the question and relevant text blocks to the OpenAI API with an absolute rule: if the answer is not in the provided text, the AI must reply exactly with: *"I don't have enough evidence to answer that."*

- ✓ **FR-3.5: Token and Budget Boundaries:** The subsystem shall enforce hard limits on both input context and output response lengths to control API token usage and prevent cost overhead.

3.2 Non-Functional Requirements

3.2.1 System Architecture & Performance

- ✓ **NFR-1.1:** Separation of Concerns: The application must maintain a strict decoupling between the Presentation Layer (Streamlit), the API Layer (FastAPI), and the AI Data Layer (RAG Engine).
- ✓ **NFR-1.2:** Asynchronous Query Handling: The backend and RAG subsystems must handle document queries asynchronously to prevent the frontend user interface from freezing during heavy AI text processing.

3.2.2 Portability, Delivery & Security

- ✓ **NFR-1.3:** Containerized Deployment: The entire application stack (frontend, backend, and database) must be orchestrated and deployable via a single Docker environment using a standard run command.
- ✓ **NFR-1.4:** Local Storage and Privacy: To maintain data privacy for sensitive investigations, all document text chunks and vector embeddings must be stored on a secure, local file system (via ChromaDB) rather than a cloud-hosted external database.
- ✓ **NFR-1.5:** Secure Credential Management: All sensitive external keys (such as the OpenAI API Key) must be loaded dynamically into the Docker container via environment variables and never hardcoded into the source code.

3.2.3 Scalability, Testing & Cost Controls

- ✓ **NFR-1.6:** Modular Testability: The core RAG processing engine must be architected independently of the network routing layer so that hallucination-guard compliance can be unit tested without running the API or UI.
- ✓ **NFR-1.7:** API Budget Enforcements: The AI engine must enforce strict parameter caps on OpenAI text requests to maintain complete cost-efficiency and operate safely within free-tier threshold limits.
- ✓ **NFR-1.8:** Python Standard Compliance: The codebase must be written using Python 3.10+ and adhere to modern coding practices (clean modular files, type hints, and tidy formatting) to facilitate engineering evaluations.